



IF3170 Inteligensi Artifisial

Institut Teknologi Bandung

Beyond Classical Search

Semester I TA 2026/2027





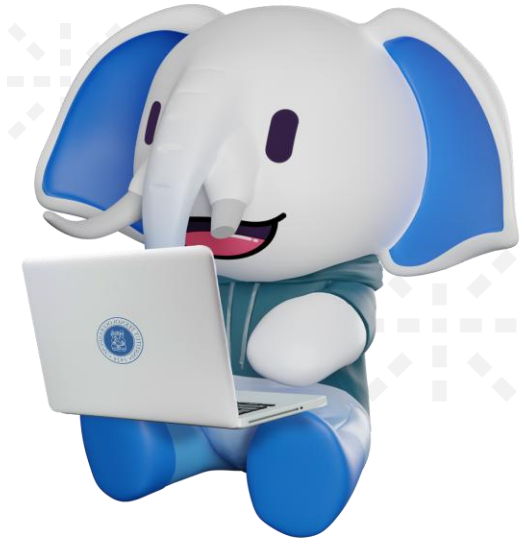
Outline

- Review: Intelligent Agent (3 types)
- Classical vs Local Search
- State: Value, Successor, and Neighbor
- Hill-climbing Search
- Simulated Annealing
- Genetic Algorithm





Classical vs Local Search





Classical Search

Problem

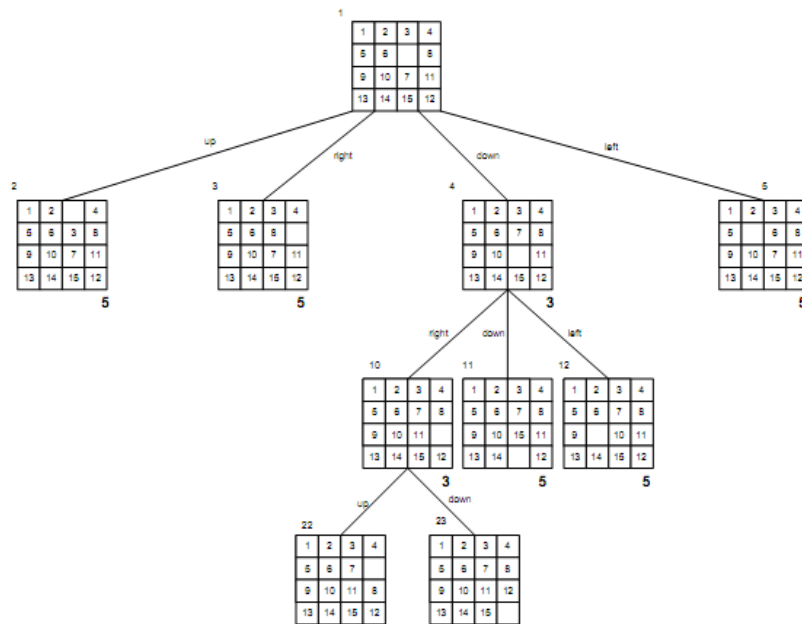
Known environment (observable, deterministic)
States (initial), operators, path cost, goal test

Explore
search space
systematically

Solution (path to goal)

Solution: sequence of actions

N-Puzzle Problem



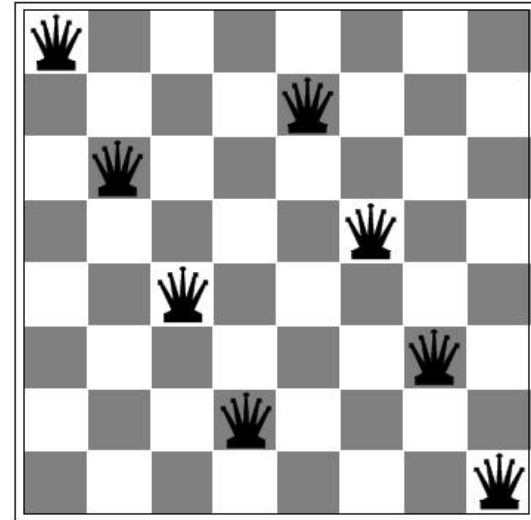
Solution: down → right → down



Path to Goal as Solution

- In many problems, the path to goal is irrelevant.
- Solution: $X=(8,6,4,2,7,5,3,1)$
- $Q1=8 \rightarrow Q2=6 \rightarrow \dots \rightarrow Q8=1$

N-Queen Problem



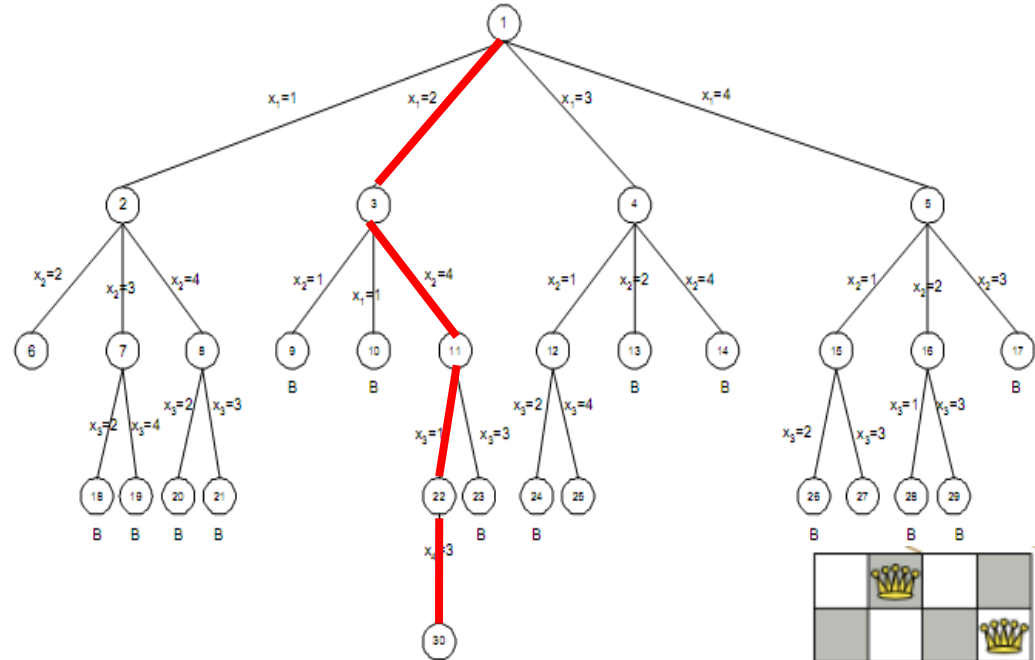
n-queens are on the board, none attacked. **one queen per column**



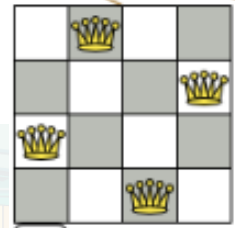
Path to Goal as Solution (2)

- For example, in n-queen
 - BFS solution:
 $x_1=2 \rightarrow x_2=4 \rightarrow x_3=1 \rightarrow x_4=3$
- What matters is the final configuration of queens, not the order in which they are added.

BFS for N-Queen Problem



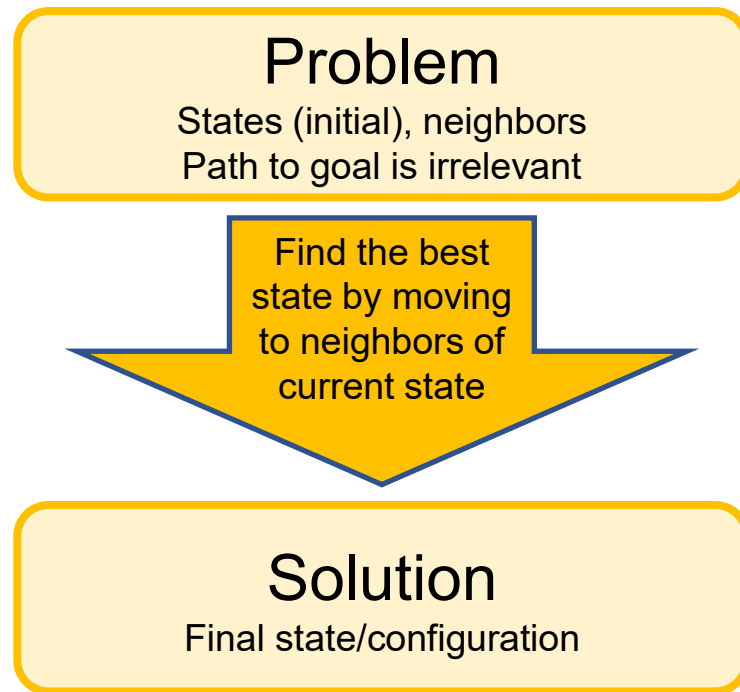
First solution 4-queen: $X=(2,4,1,3)$





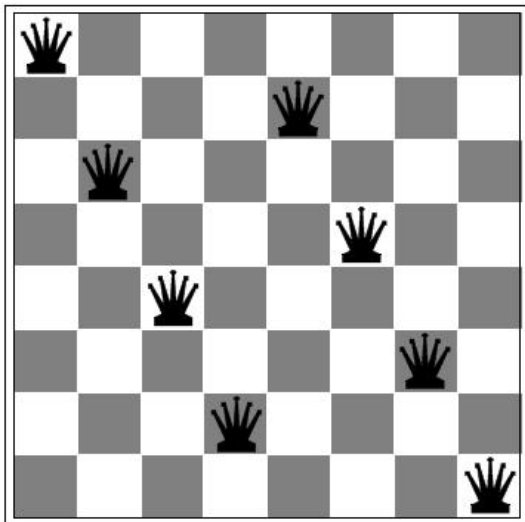
Local Search

- If path to goal does not matter, we might consider different class of algorithms.
- Local search: complete state formulation
- State: "complete" configurations
- Keep single current state, not paths.
- Action: move only to neighbors of current state.
- No path cost → state value: value according to objective function or heuristic cost function
- No goal test → maximum state value
- Solution: final state.





N-Queen Classical vs Local Search



Classical search

State: any arrangement of **0 to n** queens on the board (incremental)

Initial state: no queens on the board. Goal test: n-queens are on the board, none attacked

Action: add a queen to any empty square

Solution: path to goal

Local search

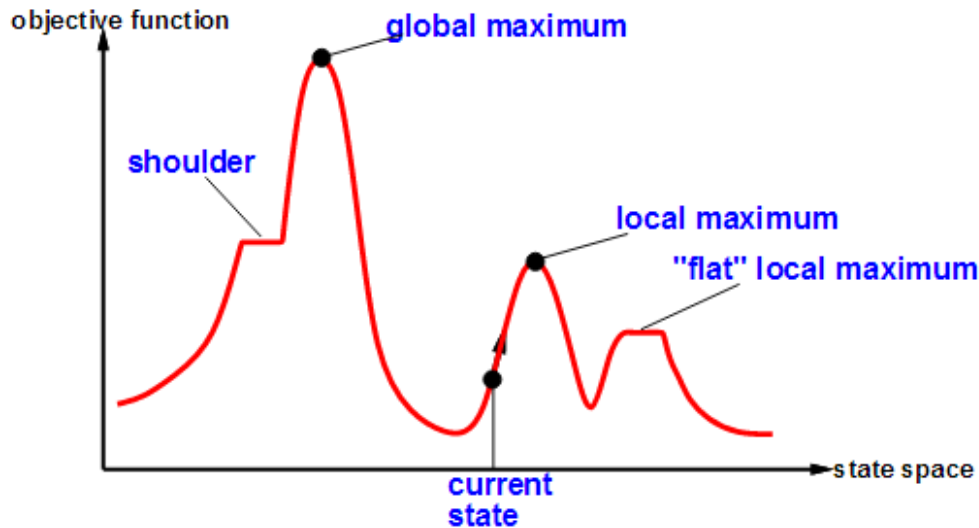
State: any arrangement of **n** queens on the board (complete)

Initial state: a [random] state. Goal test: state value = global maximum

Action: move a single queen to another square (neighbor)

Solution: final state

Local Search Explore State-space Landscape



A one-dimensional state-space landscape in which elevation corresponds to the objective function. The aim is to find the global maximum. (Russel & Norvig, 2010)

- A landscape has “location” (defined by state) and “elevation” (value of objective or heuristic cost value).
- Local search aims to find global optimum.
- Problem: depending on initial state, can get stuck in local optimum.



Summary: Local Search

Keep single current state

State: "complete" configurations

Complete formulation

Find the best state (global optimum)

Action: move to neighbors

Path to goal is irrelevant

Solution: final state

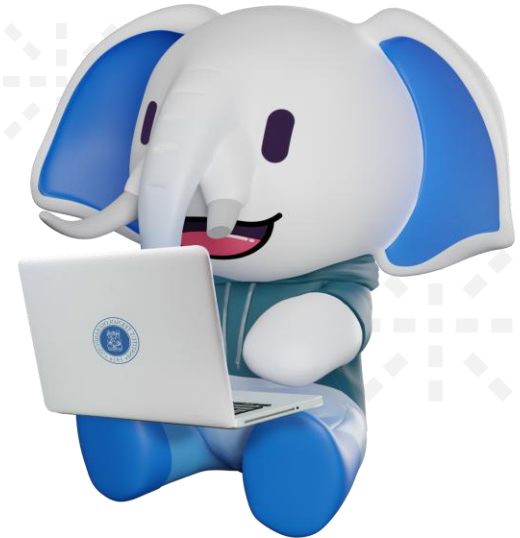
Next:

- State
- Successor
- Neighbors





State: Value, Successor, and Neighbor





Local Search: State

Keep single current state

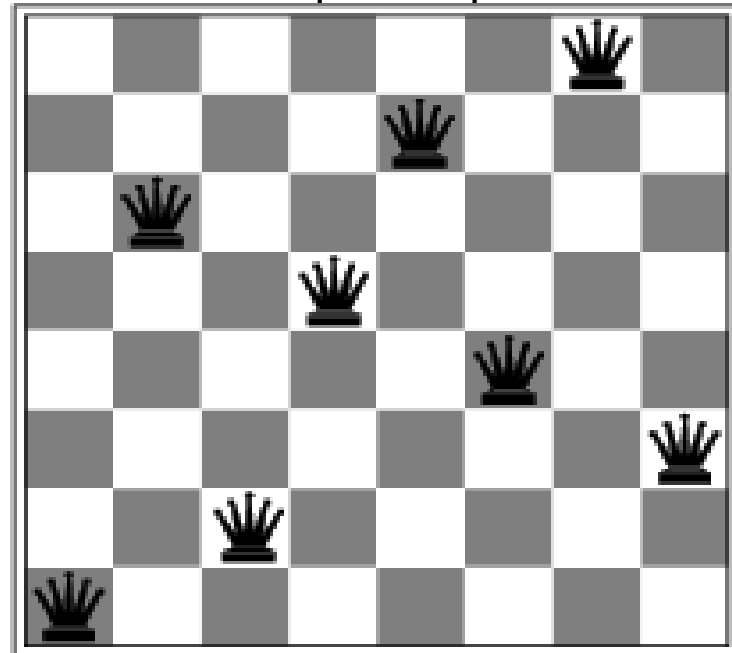
State: "complete" configurations (one queen per column)

Solution: final state

Find the best state (global optimum)

Action: move to neighbors

State for 8-queens problem



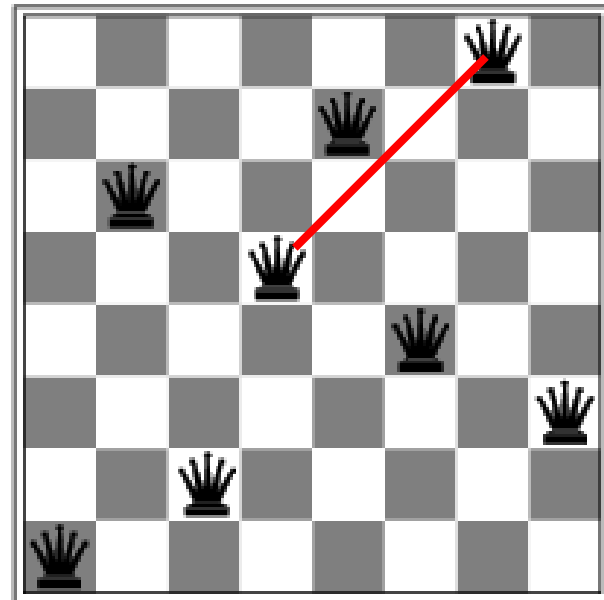
Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8

Each state has state value based on heuristic cost function.



State Value (h) for 8-queens problem

- $h = -$ number of pairs of queens that are attacking each other, either directly or indirectly.
- $h = -1$: only 1 pair of queens (Q4 attacks Q7)
- Optimum solution has global maximum, $h=0$.

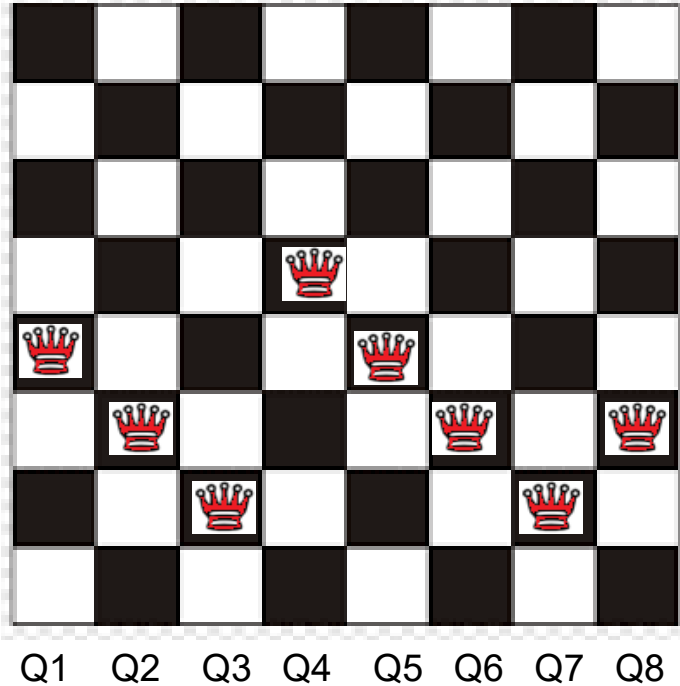


Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8

$h = -1$ for the above state



Exercise: Determine h



List pairs of queens that are attacking each other, either directly or indirectly.

h = - number of pairs of queens that are attacking each other, either directly or indirectly.



Local Search: Action

Keep single current state
State: "complete" configurations
(one queen per column)
Solution: final state

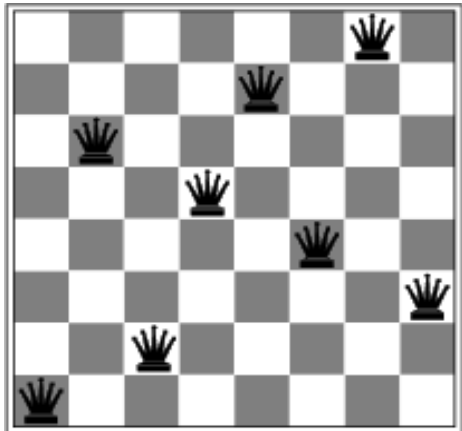
Find the best state (global optimum) ○
Action: move to neighbor ○

What is neighbor
of a state ?





Neighbor and Successors for 8-queens



Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8

Neighbor: highest-valued successor

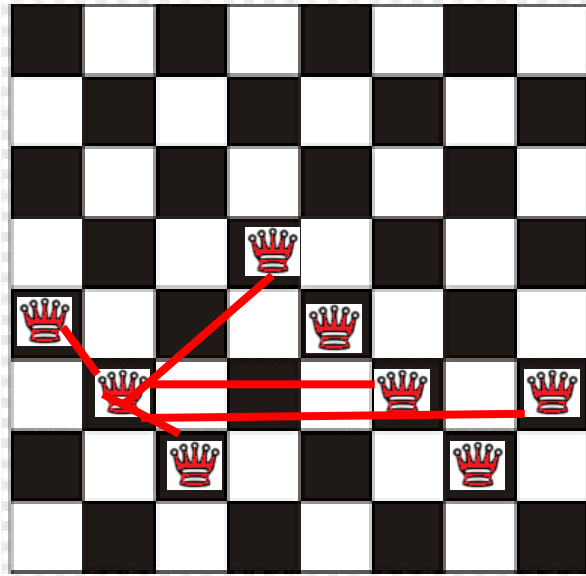
- Successor function returns all possible states generated by moving a single queen to another square in the same column.
- Each state has $8 \times 7 = 56$ successors
- Choose randomly among the set of best successors **if there is more than one**.

Neighbor: random successor

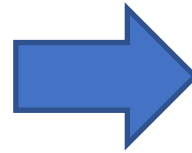
- Successor function returns a random state generated by moving a random single queen to another square in the same column.



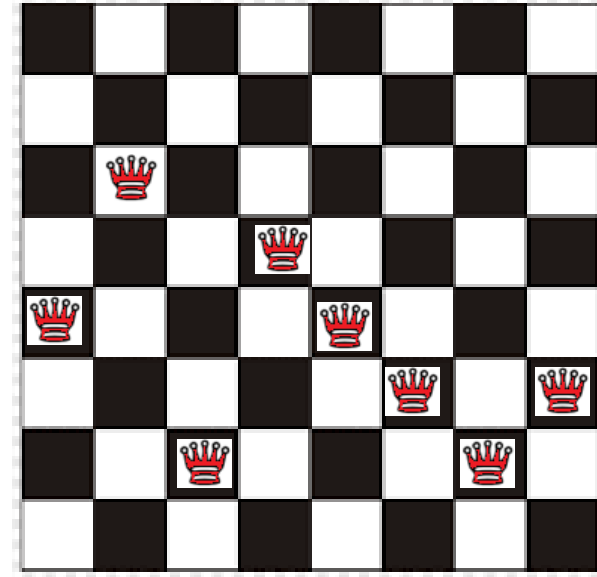
Neighbor: Random Successor



Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8



Column: Q2
Row: 3 → 6



Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8



Neighbor: Highest-valued Successor

Each number indicates h if we move a queen in its corresponding column

State value dari setiap successor belum diberi tanda negative (cost)

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♚	13	16	13	16
♚	14	17	15	♚	14	16	16
17	♚	16	18	15	♚	15	♚
18	14	♚	15	15	14	♚	16
14	14	13	17	12	14	12	18
Q1	Q2	Q3	Q4	Q5	Q6	Q7	Q8



Summary: State, Neighbor

Keep single current state
State: "complete" configurations
(one queen per column)
Solution: final state

Find the best state (global optimum)

Action: move to neighbors

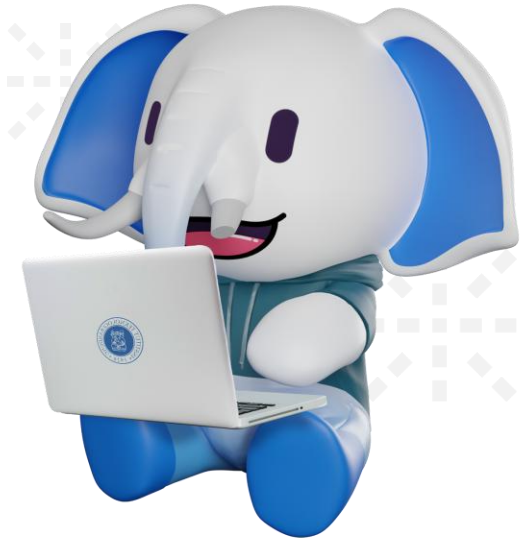
Next:

- Hill climbing Search
- Simulated annealing
- Genetic algorithm





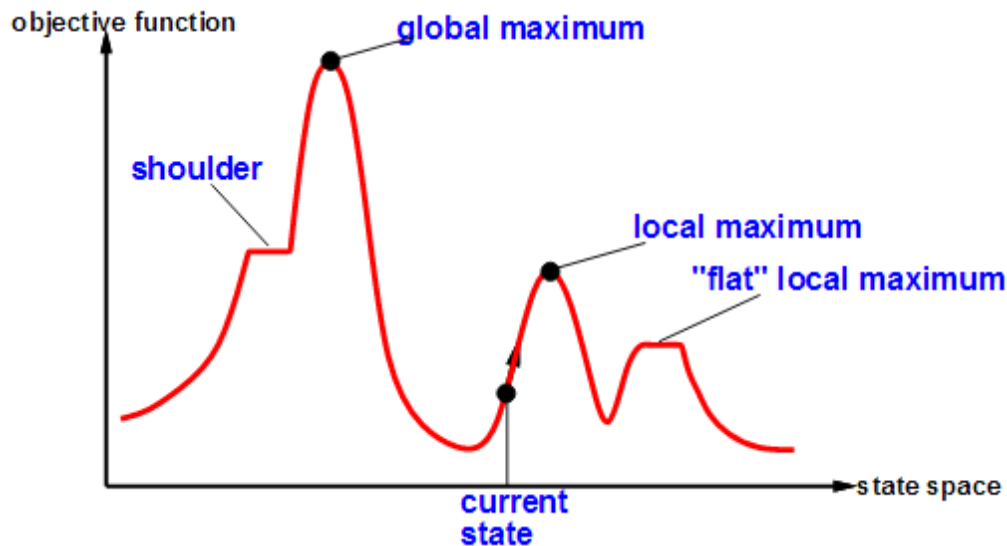
Hill-climbing Search





Hill-climbing Search

“Like climbing Everest in thick fog with amnesia”



Starting from a randomly generated initial state

Loop that continually moves in the direction of increasing value (objective) or decreasing value (cost)

Terminates when it reaches a “peak” where no neighbor has a higher value



Hill-climbing Search: Steepest Ascent

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

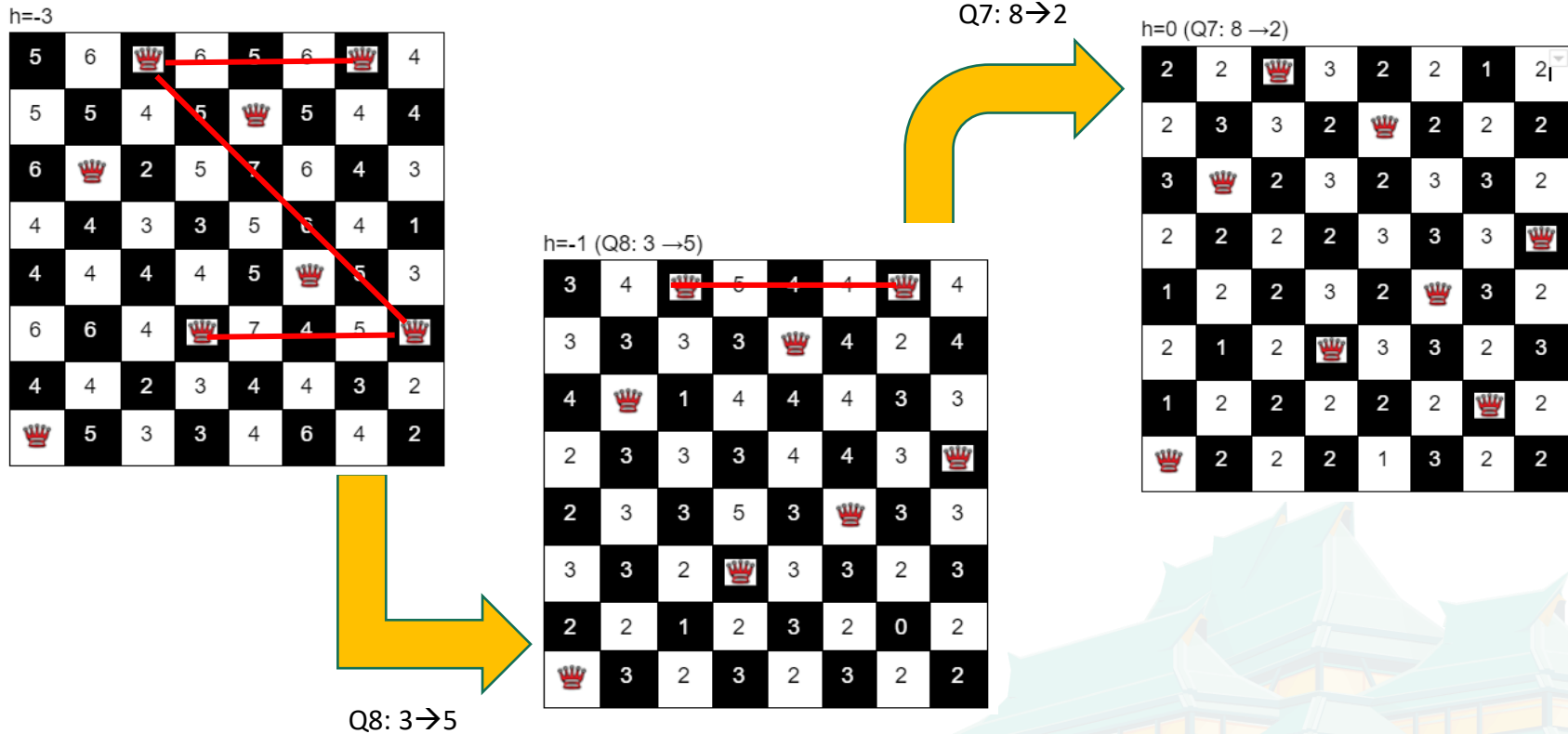
Starting from a
randomly generated
initial state

Loop that continually
moves in the direction of
increasing value (objective)
or decreasing value (cost)

Terminates when it reaches
a “**peak**” including “**flat**”
where no neighbor has a
higher value



Hill-climbing: Illustration





Hill-climbing: Stuck In Local Optimum

18	12	14	13	13	12	14	14
14	16	13	15	12	14	12	16
14	12	18	13	15	12	14	14
15	14	14	♔	13	16	13	16
♔	14	17	15	♔	14	16	16
17	♔	16	18	15	♔	15	♔
18	14	♔	15	15	14	♔	16
14	14	13	17	12	14	12	18

Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8



Step 1: $h=-17 \rightarrow h=-12$

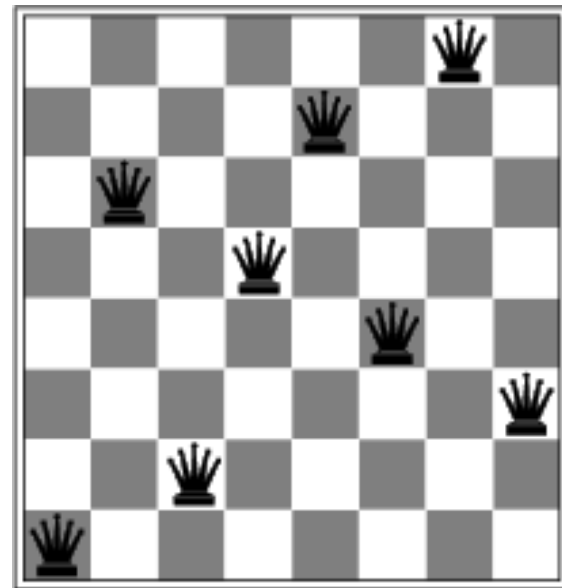
Step 2: $h=-12 \rightarrow h=-7$

Step 3: $h=-7 \rightarrow h=-4$

Step 4: $h=-4 \rightarrow h=-3$

Step 5: $h=-3 \rightarrow h=-1$

Step 6: $h=-1$ stop



Q1 Q2 Q3 Q4 Q5 Q6 Q7 Q8



Hill-climbing for 8-Queen Problem

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if *neighbor*.VALUE $\leq$ *current*.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

State space:
 $8^8 \approx 16.8$ million states

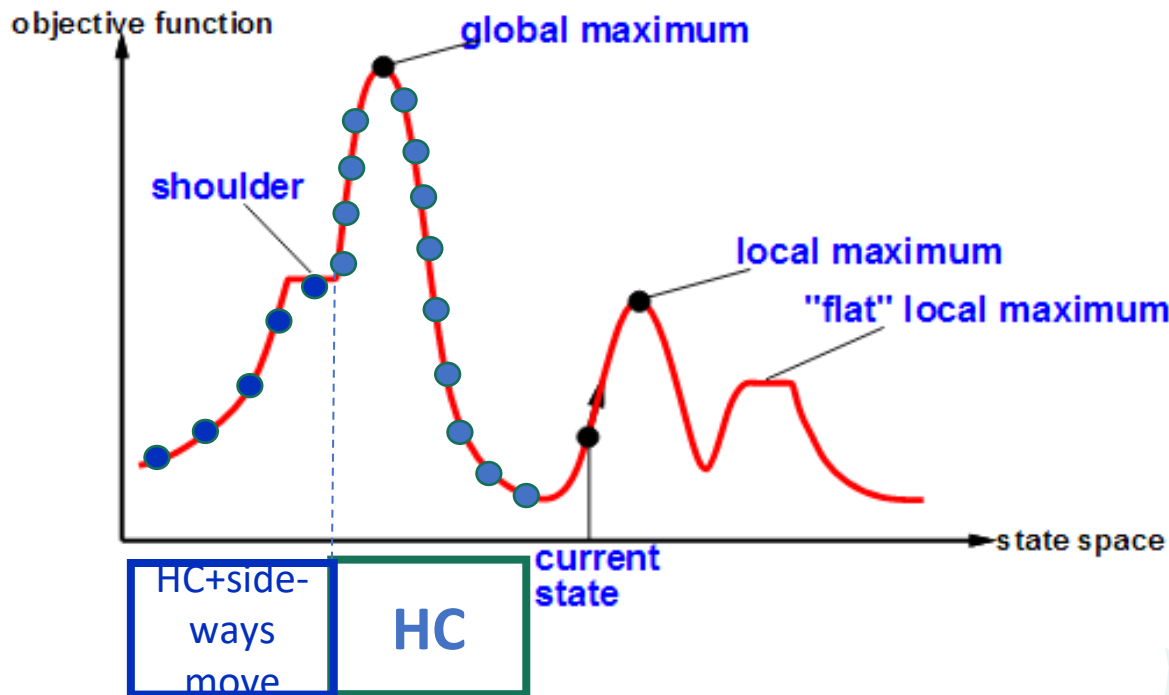
Average case: works quickly
when **success**: avg 4 steps
when **stuck**: avg 3 steps

Best case:
initial state = goal state
Prob: $92 / 8^8 = 0.00054\%$

Get **stuck** 86%, **solving**
only 14% of problem
instances



Hill-climbing Search: Final State



Success: global maximum

Stuck: 1) local maximum, 2) flat local maximum, 3) shoulder

Shoulder: still possible to global max.
Variant HC: + **sideways move** with limit on number of consecutive ways



Variant 1: Hill-climbing with Sideways Move

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

loop do

neighbor $\leftarrow$ a highest-valued successor of *current*

if neighbor.VALUE $<$ current.VALUE **then return** *current*.STATE

current $\leftarrow$ *neighbor*

Terminates when it reaches a “peak”
including “flat”

Increase success for 8-queens problem.
Limit=100:
14% $\rightarrow$ 94% success

Works **slower**:
when **success**: avg 4 $\rightarrow$ 21 steps
when **stuck**: avg 3 $\rightarrow$ 64 steps



Variant 2: Random Restart Hill-climbing

If at first you don't succeed, try, try again. It conducts **a series of hill climbing** searches (random initial states, until a goal is found)

Expected nb of restarts = $1/p \rightarrow p=0.14$: 7 restart

Expected nb of steps = $s + f(1-p)/p \rightarrow p=0.14, s=4, f=3$: 22 steps.

Very effective for n-queens problem: solving 3 million queens in under a minute (Luby et al., 1993)



Variant 3: Stochastic Hill-climbing

function HILL-CLIMBING(*problem*) **returns** a state that is a local maximum

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

repeat *nmax* **times**

neighbor $\leftarrow$ a **random** successor of *current*

if *neighbor*.VALUE $\triangleright$ *current*.VALUE **then return** *current* $\leftarrow$ *neighbor*

Terminates
when it reaches
nmax iteration

Generating a
successor randomly
(**not all** successor)

Move to neighbor
if it is better than
current state.

Works **slower**
(more steps)



Summary: Hill-climbing

continually moves in the direction of increasing value (objective) or decreasing value (cost)

Depending on initial state, can get stuck in local maxima.

Next:

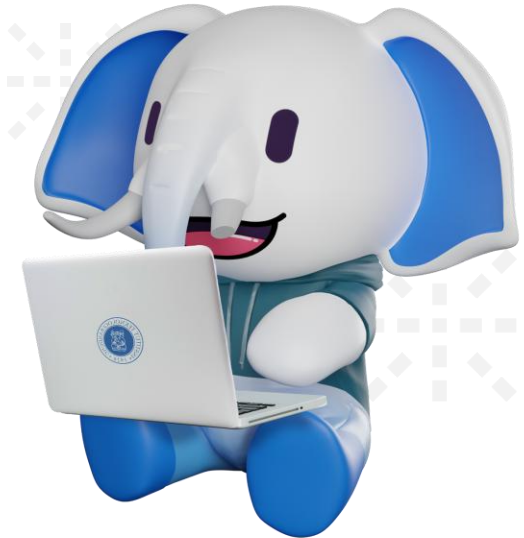
- Simulated annealing

Variant: steepest ascent HC, HC with sideways move, stochastic HC, random restart HC



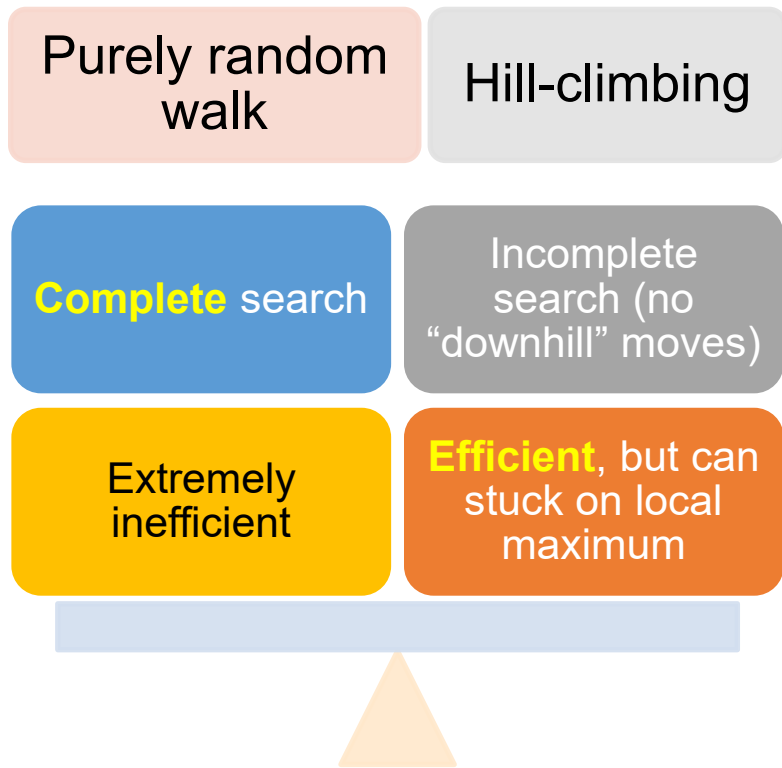


Simulated Annealing





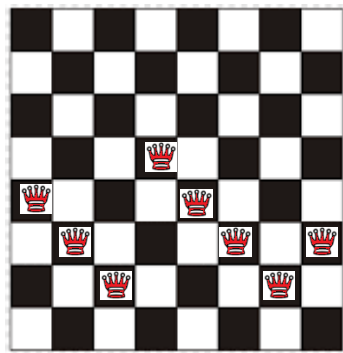
SA: Combining Completeness and Efficiency



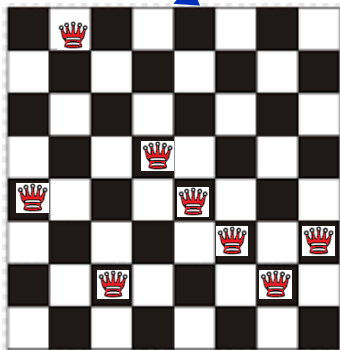
- Simulated annealing combines hill climbing (efficient) with a random walk (complete)
- Idea: escape local maxima by **allowing some "bad" moves** but **gradually decrease** their frequency
- Simulated annealing is a version of **stochastic hill climbing** where **some downhill moves are allowed**.



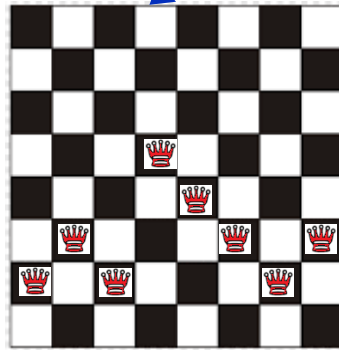
Neighbor: One Random Successor of Current



56 possible random successor



Better neighbor: $h=-12$



Worse neighbor: $h=-18$

- Stochastic hill-climbing:
 - Only move to better neighbor, and skip worse neighbor.
- Simulated annealing:
 - move to better neighbor, and allow move to worse neighbor that has probability $e^{\Delta E/T}$



Simulated Annealing



https://id.m.wikipedia.org/wiki/Berkas:Annealing_a_silver_strip.JPG

Annealing: heat (metal or glass) and allow it to cool slowly, in order to remove internal stresses and toughen it.

- T is the “temperature” of annealing that gradually decreases.
- $\Delta E = \text{neighbor.value} - \text{current.value}$
- If better neighbor ($\Delta E > 0$): move to neighbor (**stochastic HC**)
- If worse neighbor ($\Delta E < 0$): probability move
 - When T is high (e.g. $\Delta E = -5$, $T = 100$, $\text{prob} = 0.95$), there is a lot of random motion → **random walk**
 - When T approaches 0 (e.g. $\Delta E = -5$, $T = 1$, $\text{prob} = 0.007$), randomness is decreased → **stochastic hill climbing**.



Simulated Annealing (Russel & Norvig, 2010)

function SIMULATED-ANNEALING(*problem*, *schedule*) **returns** a solution state

inputs: *problem*, a problem

schedule, a mapping from time to “temperature”

current $\leftarrow$ MAKE-NODE(*problem*.INITIAL-STATE)

for $t = 1$ **to** ∞ **do**

T $\leftarrow$ *schedule*(*t*)

if $T = 0$ **then return** *current*

next $\leftarrow$ a randomly selected successor of *current*

$\Delta E \leftarrow next.VALUE - current.VALUE$

if $\Delta E > 0$ **then** *current* $\leftarrow$ *next*

else *current* $\leftarrow$ *next* only with probability $e^{\Delta E/T}$

T: temperature
as a function of
time *t*

Terminates
 $T=0$

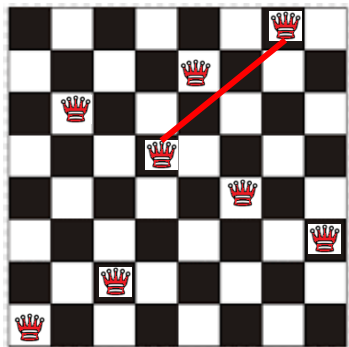
Move to better
neighbor
(stochastic HC)

allowing some "bad"
moves, depends on
probability

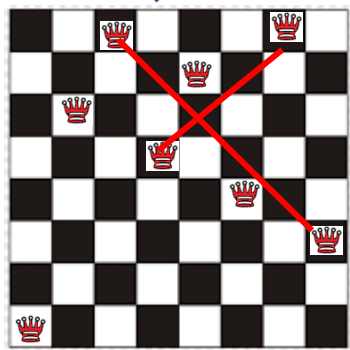


Move to Worse State

Current State: $h=-1$



Column: C3
Row: 2 $\rightarrow$ 8



$current \leftarrow next$ only with probability $e^{\Delta E/T}$

Compare to static value:
move probability > 0.5

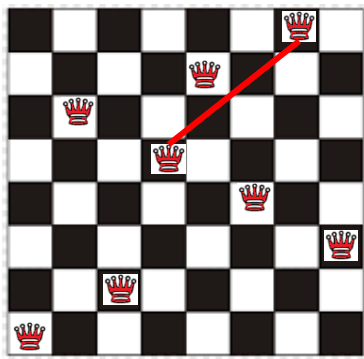
Compare to random value:
move probability $> \text{random}(0,1)$

Next: $h=-2$, $T=10$

$\Delta E < 0$: prob = $e^{(-2-(-1))/10} = e^{-0.1} = 0.9$

Simulated Annealing: Illustration

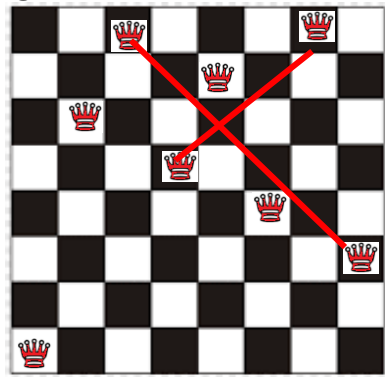
Current State: $h=-1$



Column: C4
Row: 5 $\rightarrow$ 3



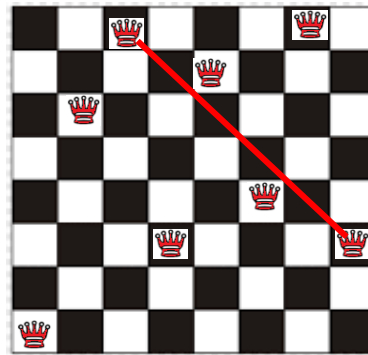
Next: $h=-2$, $T=10$
 $\text{Prob} = e^{(-2 - (-1))/10} = e^{-0.1} = 0.9$
 Current $\leftarrow$ next



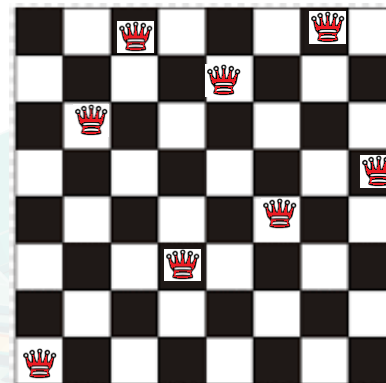
Column: C3
Row: 2 $\rightarrow$ 8



Next: $h=-1$
 Current $\leftarrow$ next



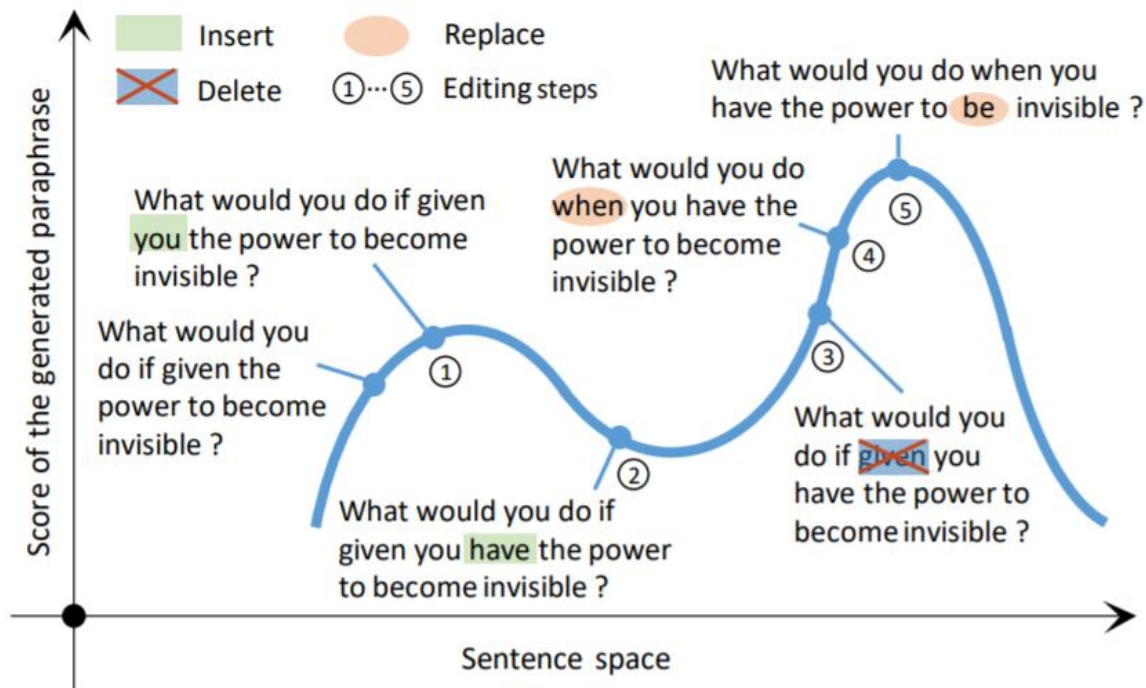
Next: $h=0$
 Current $\leftarrow$ next



Column: C8
Row: 3 $\rightarrow$ 5



Application: Simulated Annealing for Paraphrase



Liu, X., Mou, L., Meng, F., Zhou, H., Zhou, J., & Song, S. (2020). Unsupervised paraphrasing by simulated annealing. Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics, pages 302–312 July 5 - 10, 2020. c 2020 Association for Computational Linguistics. *arXiv preprint arXiv:1909.03588*.



Properties of Simulated Annealing Search



One can prove: If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1



Widely used in VLSI layout, airline scheduling, etc





Summary: Simulated Annealing

Simulated annealing is a version of stochastic hill climbing where some downhill moves are allowed.

If T decreases slowly enough, then simulated annealing search will find a global optimum with probability approaching 1

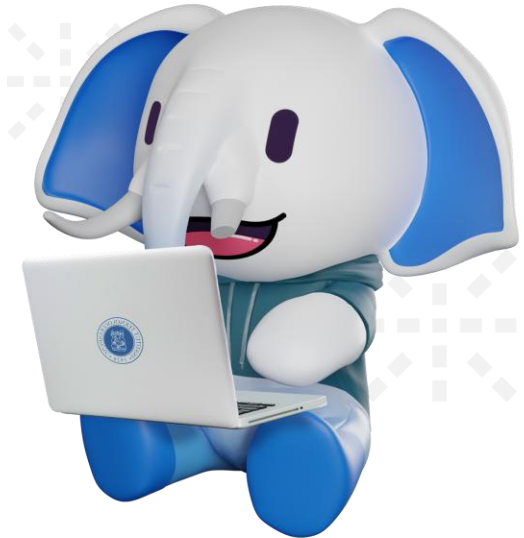
Next:

- Genetic Algorithm



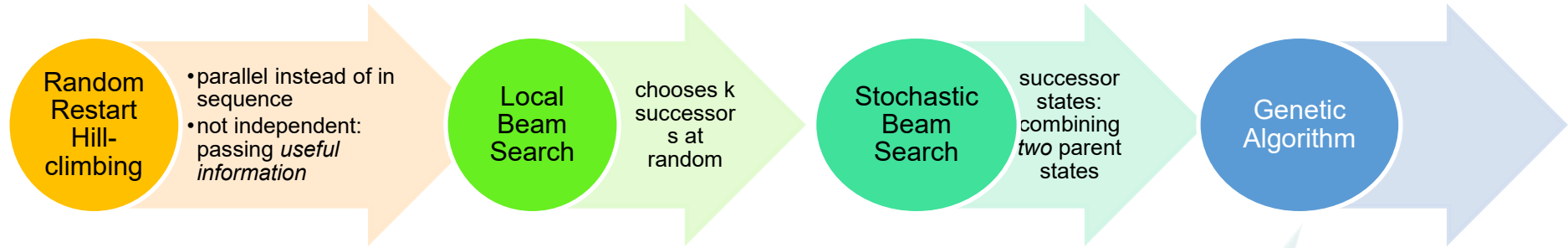


Genetic Algorithm





Genetic Algorithm



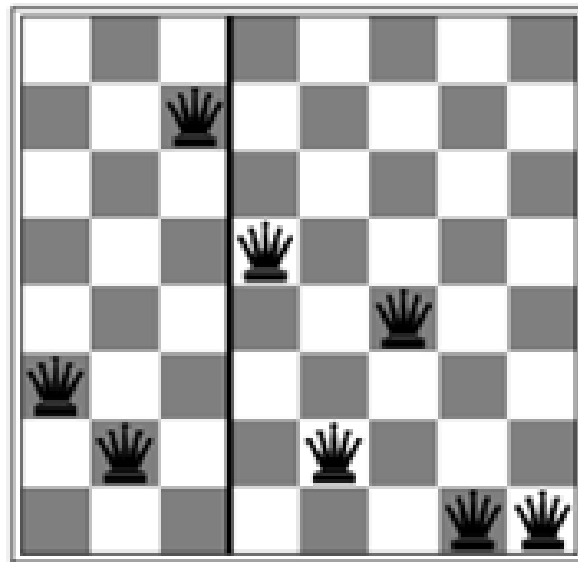


State / Individual

A state is represented as a string over a finite alphabet (often a string of 0s and 1s)

One character ~ one variable

32752411

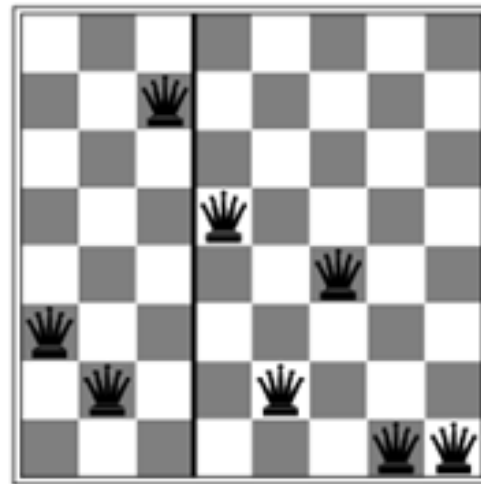




State Value: Fitness Function

- Evaluation function (**fitness function**).
- Higher values for better states.
- Fitness function for n-queen:
number of **non-attacking pairs** of queens
 - $\min = 0$
 - $\max = (8 \times 7) / 2 = 28$
(**global maximum**)

32752411



$$F(32752411) = 28 - 5 = 23$$



Initial Population & Successor Function

- GA starts with k randomly generated states.
- A successor state is generated by combining two parent states
- Produce the next generation of states by selection, crossover, and mutation

24748552

• $F=24$

32752411

• $F=23$

24415124

• $F=20$

32543213

• $F=11$

Random Selection of Parent States

Probability of random selection

24748552

- $24/(24+23+20+11)$
- = 31%

32752411

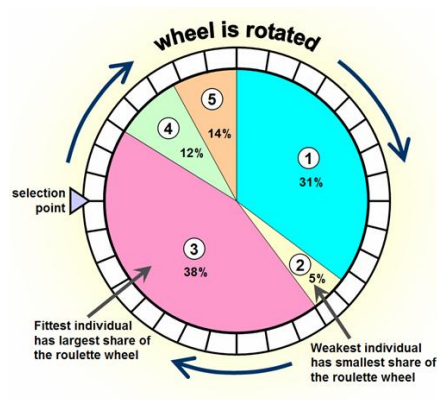
- $23/(24+23+20+11)$
- = 29%

24415124

- $20/(24+23+20+11)$
- = 26%

32543213

- $11/(24+23+20+11)$
- = 14%



Parents selected

32752411

24748552

0-31

32-60

61-86

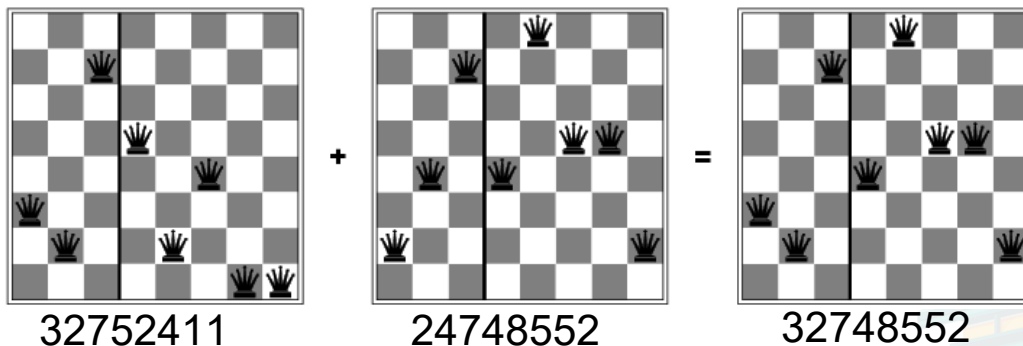
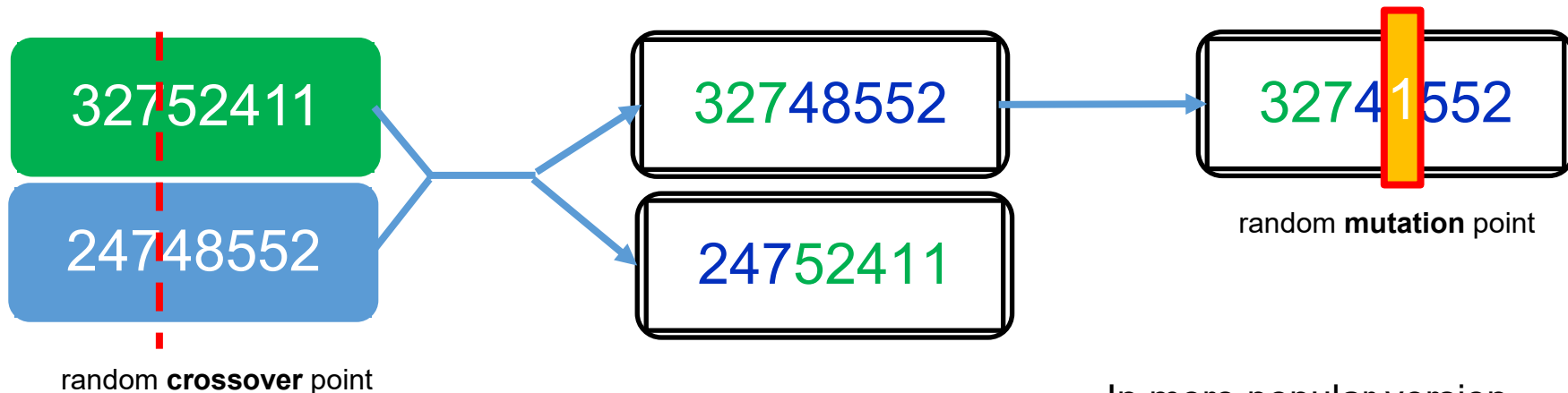
87-100

32752411

24415124

random(0,100)

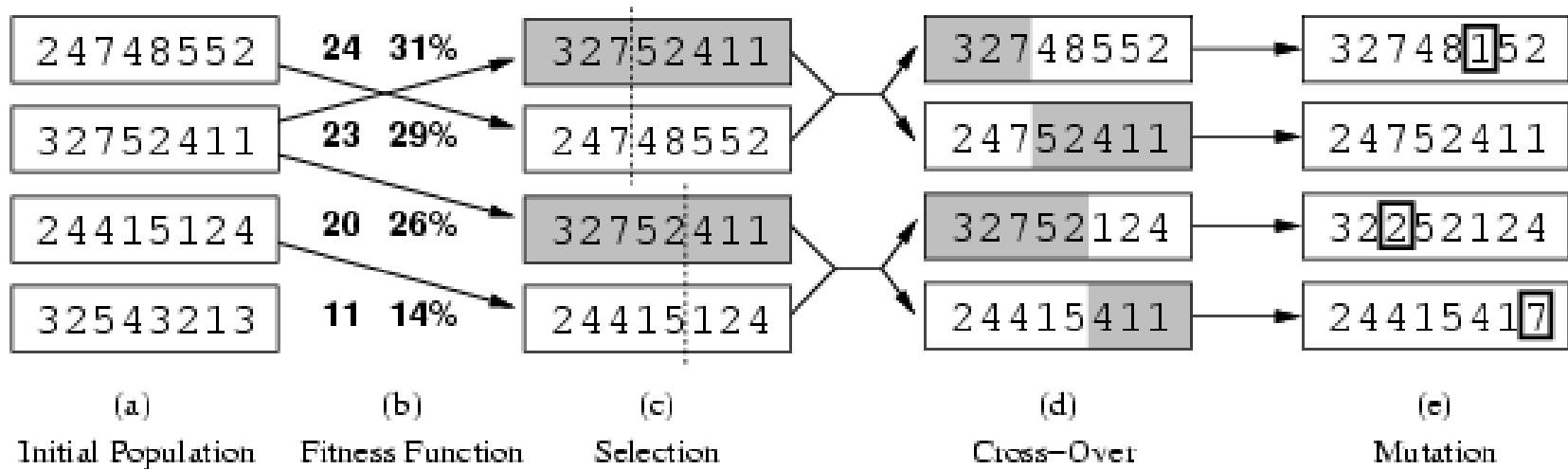
Cross Over / Reproduce & Mutation



In more popular version, each mating of two parents produces only one offspring, not two (Russel & Norvig, 2010)



Genetic Algorithm: Illustration (Russel & Norvig, 2010)





Genetic Algorithm

function GENETIC-ALGORITHM(*population*, FITNESS-FN) **returns** an individual

inputs: *population*, a set of individuals

FITNESS-FN, a function that measures the fitness of an individual

repeat

new_population $\leftarrow$ empty set

for $i = 1$ **to** SIZE(*population*) **do**

x $\leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

y $\leftarrow$ RANDOM-SELECTION(*population*, FITNESS-FN)

child $\leftarrow$ REPRODUCE(*x*, *y*)

if (small random probability) **then** *child* $\leftarrow$ MUTATE(*child*)

add *child* to *new_population*

population $\leftarrow$ *new_population*

until some individual is fit enough, or enough time has elapsed

return the best individual in *population*, according to FITNESS-FN



Summary

k randomly generated states (population: k individual)

Fitness function (state value)

successor function: combining two parent states (selection, cross-over, mutation)



Thank You

